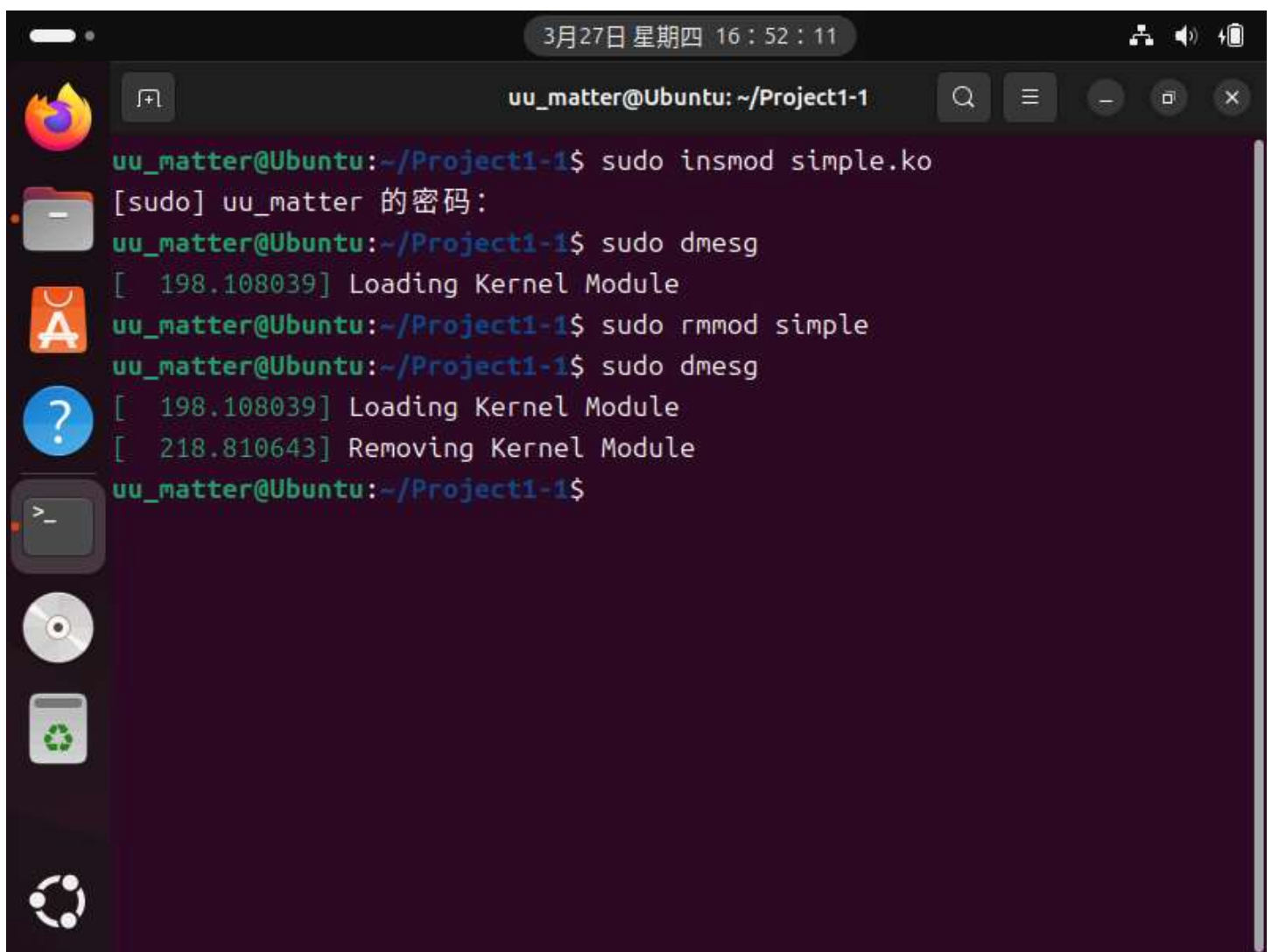


Project 1 Report

1-1 内核模块概述

下载OSC10E配套代码，转到源代码文件夹下使用命令 `make` 编译得到 `simple.ko`，使用命令 `sudo insmod simple.ko` 将模块插入内核，使用命令 `sudo dmesg` 查看内核日志缓冲区，发现内核日志 `Loading Kernel Module`，说明内核模块已加载；使用命令 `sudo rmmod simple` 移除内核模块，再次使用命令 `sudo dmesg` 查看内核日志缓冲区，发现内核日志 `Removing Kernel Module`，说明内核模块已被移除

A terminal window titled 'uu_matter@Ubuntu: ~/Project1-1' with a timestamp '3月27日 星期四 16:52:11'. The terminal shows the following commands and output:

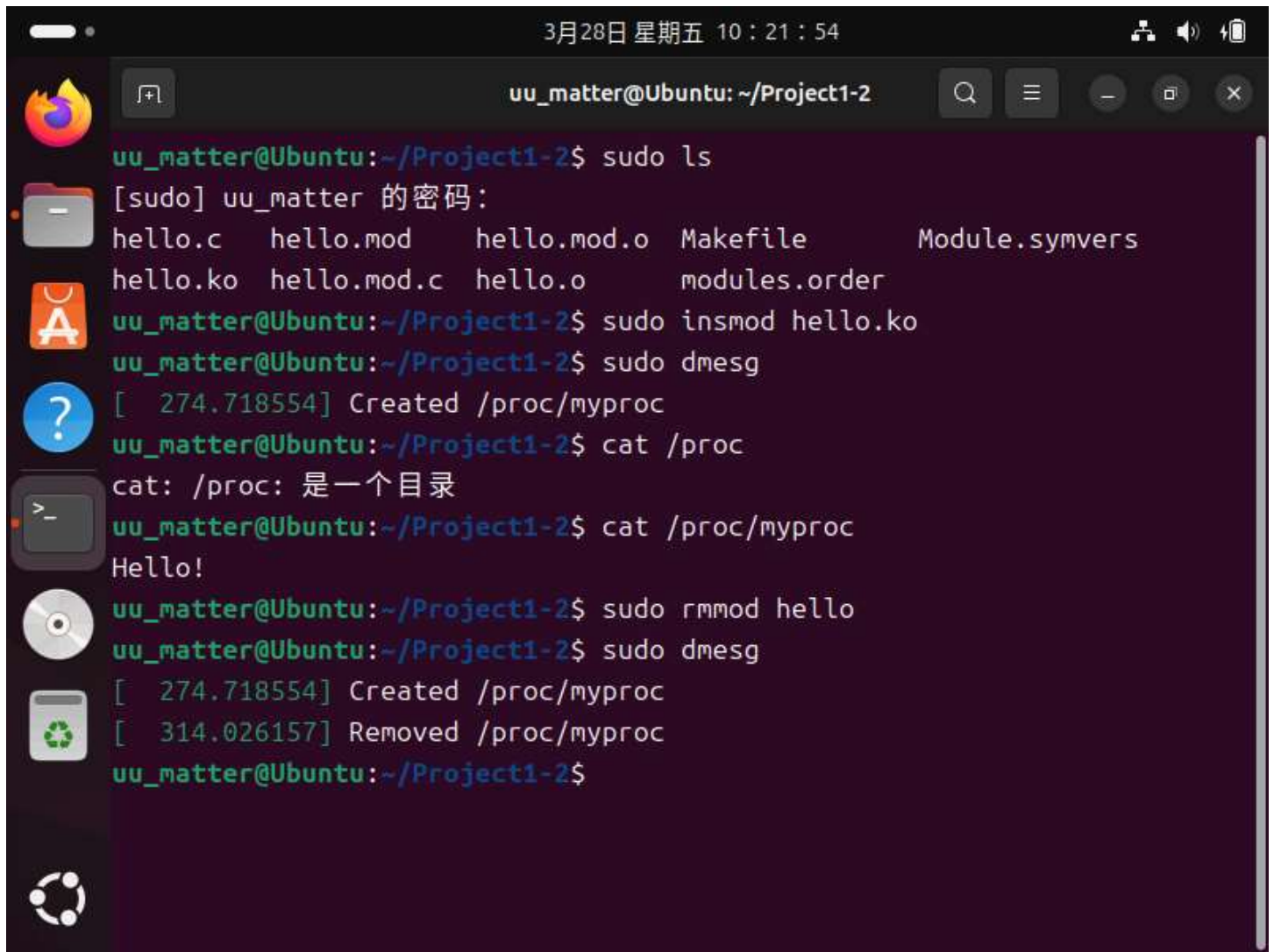
```
uu_matter@Ubuntu:~/Project1-1$ sudo insmod simple.ko
[sudo] uu_matter 的密码:
uu_matter@Ubuntu:~/Project1-1$ sudo dmesg
[ 198.108039] Loading Kernel Module
uu_matter@Ubuntu:~/Project1-1$ sudo rmmod simple
uu_matter@Ubuntu:~/Project1-1$ sudo dmesg
[ 198.108039] Loading Kernel Module
[ 218.810643] Removing Kernel Module
uu_matter@Ubuntu:~/Project1-1$
```

源码 Source Code\Project1-1\simple.c

1-2 /proc文件系统概述

由于OSC10E配套代码已不适用于最新Linux内核，故采用高版本API
将 `hello.ko` 插入内核后，使用命令 `cat /proc`，命令行提示 `cat: /proc: 是一个目录`，使用命

令 `cat /proc/myproc`，得到信息 `Hello!`，说明目录和文件已创建

A terminal window titled 'uu_matter@Ubuntu: ~/Project1-2' with a dark purple background. The window shows a series of commands and their outputs. The user runs 'sudo ls' to list files in the current directory, including 'hello.c', 'hello.mod', 'hello.mod.o', 'Makefile', 'Module.symvers', 'hello.ko', 'hello.mod.c', 'hello.o', and 'modules.order'. Then, they run 'sudo insmod hello.ko' to load the module. Next, 'sudo dmesg' is run, showing a kernel message: '[274.718554] Created /proc/myproc'. The user then runs 'cat /proc' and receives the error 'cat: /proc: 是一个目录'. Finally, they run 'cat /proc/myproc' and get the output 'Hello!'. After that, they run 'sudo rmmod hello' to remove the module. Another 'sudo dmesg' shows two messages: '[274.718554] Created /proc/myproc' and '[314.026157] Removed /proc/myproc'. The terminal ends with the prompt 'uu_matter@Ubuntu: ~/Project1-2\$'.

```
uu_matter@Ubuntu:~/Project1-2$ sudo ls
[sudo] uu_matter 的密码:
hello.c  hello.mod  hello.mod.o  Makefile      Module.symvers
hello.ko  hello.mod.c  hello.o      modules.order

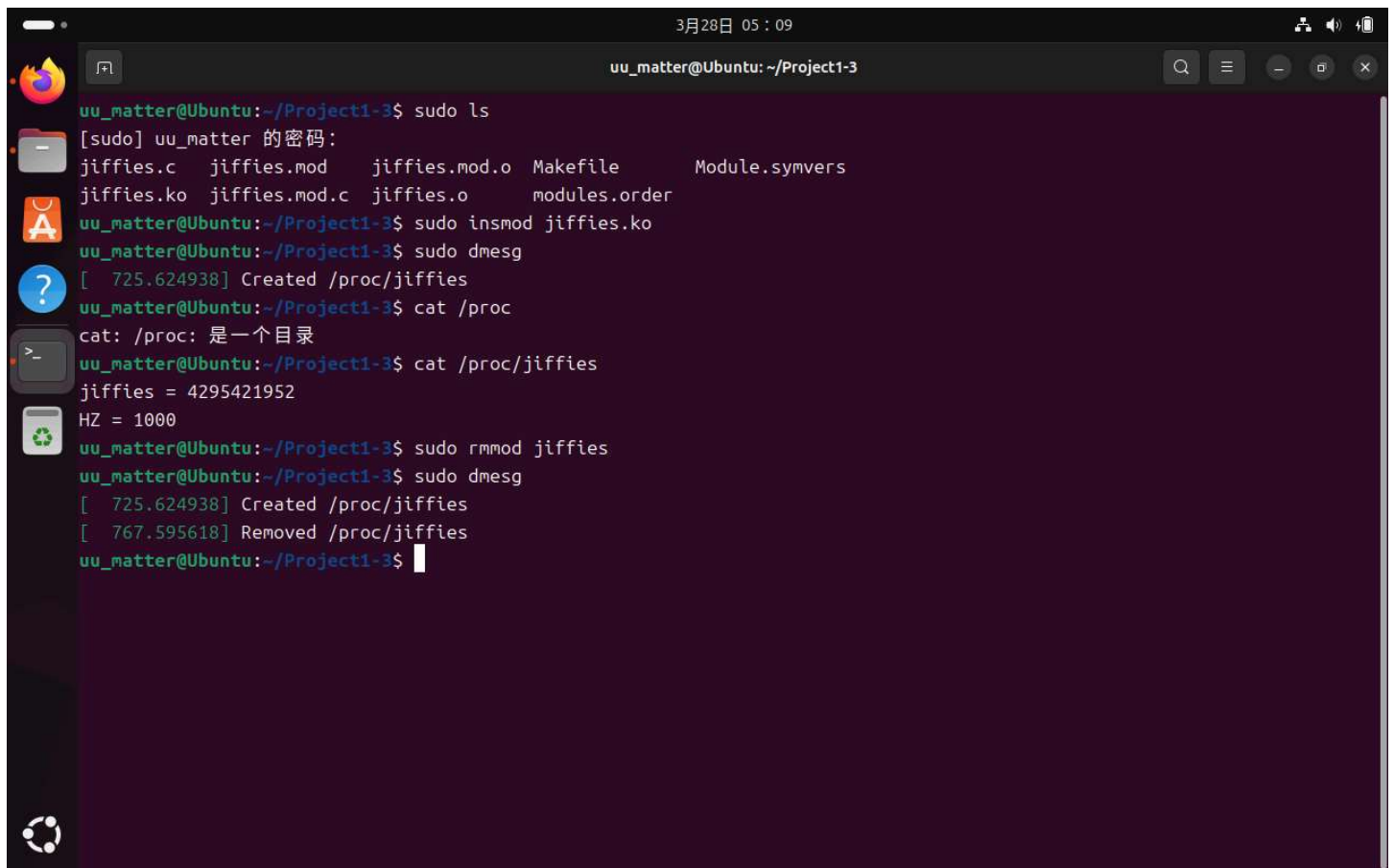
uu_matter@Ubuntu:~/Project1-2$ sudo insmod hello.ko
uu_matter@Ubuntu:~/Project1-2$ sudo dmesg
[ 274.718554] Created /proc/myproc
uu_matter@Ubuntu:~/Project1-2$ cat /proc
cat: /proc: 是一个目录
uu_matter@Ubuntu:~/Project1-2$ cat /proc/myproc
Hello!
uu_matter@Ubuntu:~/Project1-2$ sudo rmmod hello
uu_matter@Ubuntu:~/Project1-2$ sudo dmesg
[ 274.718554] Created /proc/myproc
[ 314.026157] Removed /proc/myproc
uu_matter@Ubuntu:~/Project1-2$
```

源码 `Source Code\Project1-2\hello.c`

1-3 jiffies和HZ的值

将1-2中的路径 `myproc` 改为 `jiffies`，将报告函数中的输出提示信息 `Hello!` 改为输出变量的值 `jiffies` 与 `HZ` 即可

将 `jiffies.ko` 插入内核后，使用命令 `cat /proc/jiffies`，得到信息 `jiffies` 与 `HZ` 的值，说明功能已实现



```
uu_matter@Ubuntu: ~/Project1-3
[ 725.624938] Created /proc/jiffies
uu_matter@Ubuntu: ~/Project1-3$ cat /proc/jiffies
jiffies = 4295421952
HZ = 1000
[ 767.595618] Removed /proc/jiffies
```

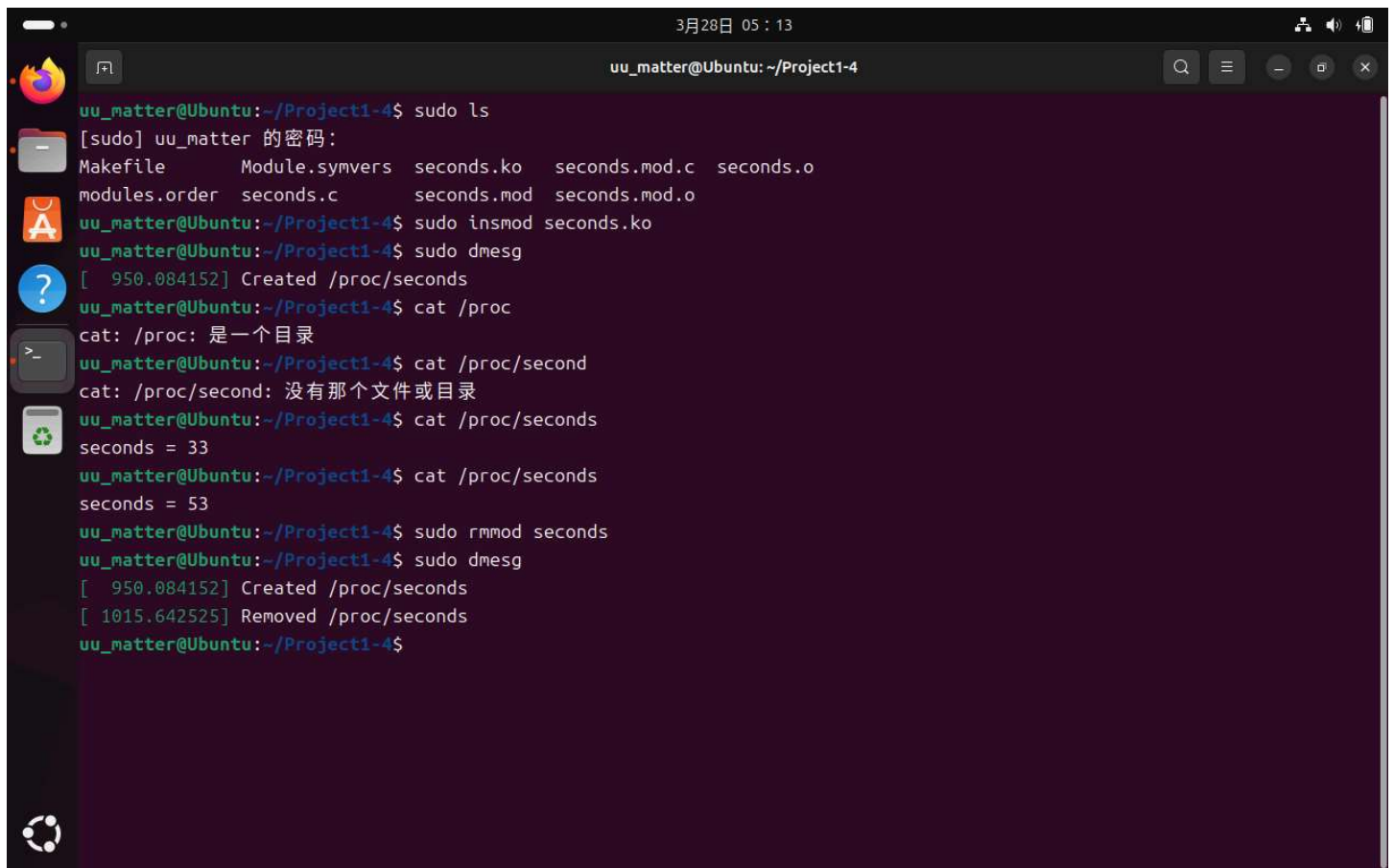
The screenshot shows a terminal window titled 'uu_matter@Ubuntu: ~/Project1-3'. The user runs 'sudo ls' and lists files: jiffies.c, jiffies.mod, jiffies.mod.o, Makefile, Module.symvers, jiffies.ko, jiffies.mod.c, jiffies.o, and modules.order. Then, they run 'sudo insmod jiffies.ko', 'sudo dmesg' (showing '[725.624938] Created /proc/jiffies'), 'cat /proc/jiffies' (showing 'jiffies = 4295421952' and 'HZ = 1000'), 'sudo rmmod jiffies', and another 'sudo dmesg' (showing '[767.595618] Removed /proc/jiffies').

源码 Source Code\Project1-3\jiffies.c

1-4 内核模块自加载以来的秒数

由于 `jiffies` 变量跟踪的是自系统启动以来发生定时器中断的数量，因此要记录内核加载时刻的 `jiffies`，计算加载模块时使用命令时 `jiffies` 的差值，并除以 `HZ`，以此方法得到秒数

将 `seconds.ko` 插入内核后，使用命令 `cat /proc/seconds`，得到信息自模块插入内核以来的秒数，再次使用命令 `cat /proc/seconds`，再次得到信息自模块插入内核以来的秒数，两次得到秒数的差值为两次使用命令的时间间隔，说明功能已实现

A terminal window titled 'uu_matter@Ubuntu: ~/Project1-4' with a search bar and window controls. The terminal shows a series of commands and their outputs. The user runs 'sudo ls' to list files in the current directory, including 'Makefile', 'Module.symvers', 'seconds.ko', 'seconds.mod.c', 'seconds.o', 'modules.order', 'seconds.c', 'seconds.mod', and 'seconds.mod.o'. Then, they run 'sudo insmod seconds.ko' to load the module. Next, 'sudo dmesg' is used to view kernel messages, showing '[950.084152] Created /proc/seconds'. The user then runs 'cat /proc' and 'cat /proc/second', receiving messages that '/proc' is a directory and '/proc/second' does not exist. Finally, they run 'cat /proc/seconds', which shows 'seconds = 33'. After running 'sudo rmod seconds', 'sudo dmesg' again shows the removal of the module at '[1015.642525] Removed /proc/seconds'.

源码 Source Code\Project1-4\seconds.c

1-5 Bonus: copy_to_user()和memcpy()的差异

copy_to_user() 和 memcpy() 都是用于内存拷贝的函数，但它们在Linux内核中有重要区别：

主要差异

使用场景

copy_to_user()：用于从内核空间拷贝数据到用户空间； memcpy()：用于在内核空间内部或用户空间内部进行内存拷贝

安全性检查

copy_to_user() 会验证目标用户空间地址是否有效且可写； memcpy() 不进行任何安全检查，假设调用者已确保参数正确

返回值

copy_to_user() 返回未能成功拷贝的字节数（成功时返回0）； memcpy() 返回目标地址指针

上下文要求

`copy_to_user()` 只能在用户上下文调用（如系统调用处理中）； `memcpy()` 可以在任何上下文中使用

页错误处理

`copy_to_user()` 能正确处理用户空间可能引发的页错误； `memcpy()` 假设内存已经有效映射

总结

差异主要在于： `copy_to_user()` 安全性较好，性能较差； `memcpy()` 则恰恰相反。在内核级别的编程中，应当使用 `copy_to_user()` 以拷贝数据到用户空间，目的是保证内核的安全